



دانشگاه صنعتی شریف
دانشکده مهندسی کامپیوتر

عنوان:

کنترل توسط برنامه ذخیره شده در حافظه

نام درس

آزمایشگاه معماری کامپیوتر

نیم سال ۱۴۰۴۳

استاد

دکتر سربازی آزاد

گروه ۲

امیرشایان سلیمانی نیا - ۴۰۳۱۰۶۱۰۸

محمد مهدی مرادی - ۴۰۳۱۰۶۶۸۱

فهرست مطالب

۱	هدف آزمایش	۲
۲	شرح آزمایش و مشخصات طراحی	۲
۳	قالب دستور و کدگذاری	۲
۴	تغییر مسیر داده نسبت به آزمایش ششم	۳
۵	طراحی واحد محاسبات	۳
۶	طراحی فایل ثبات	۴
۷	طراحی مسیر داده	۵
۸	واحد واکنشی دستور	۶
۱۰.۸	شمارنده برنامه	۶
۲.۸	حافظه برنامه	۶
۳.۸	ثبات دستور	۶
۹	مدار نهایی	۷
۱۰	برنامه تولید سری فیبوناچی	۷
۱۱	تست و شبیه سازی	۸
۱.۱۱	روند اجرای دستورهای ابتدایی	۸
۲.۱۱	نتیجه نهایی برنامه	۹
۱۲	نتیجه گیری	۱۰

۱ هدف آزمایش

هدف از این آزمایش، آشنایی با نحوه واکشی و اجرای ترتیبی دستورات ذخیره شده در حافظه برنامه است. در آزمایش ششم، واحد محاسبات و مجموعه ثبات های عمومی ماشین طراحی شد و دستورات شش بیتی به صورت مستقیم به مسیر داده اعمال می شدند. در این آزمایش، با اضافه کردن شمارنده برنامه، حافظه برنامه و ثبات دستور، ورودی دستی دستور حذف شده و دستورات به صورت خودکار و بر اساس یک برنامه ذخیره شده در حافظه واکشی و اجرا می شوند.

در پیاده سازی انجام شده، حافظه برنامه به صورت یک ROM با ظرفیت ۳۲ کلمه ۸ بیتی در نظر گرفته شده است. شمارنده برنامه یا PC پنج بیتی، آدرس کلمه بعدی را تولید می کند و خروجی حافظه پس از ذخیره شدن در ثبات دستور IR به مسیر داده اعمال می شود. برای بررسی عملکرد سیستم، برنامه تولید ده جمله اول سری فیبوناچی در حافظه قرار داده شده و اجرای مرحله به مرحله آن در محیط Logisim-Evolution بررسی شده است.

۲ شرح آزمایش و مشخصات طراحی

معماری این آزمایش از دو بخش اصلی تشکیل شده است. بخش نخست همان مسیر داده توسعه یافته آزمایش ششم است که شامل چهار ثبات عمومی هشت بیتی، واحد جمع/تفریق و مالتی پلکسهای انتخاب عملوند است. بخش دوم واحد واکشی دستور است که از PC، حافظه برنامه و IR تشکیل می شود. جریان کلی اجرای یک دستور به صورت زیر است:

PC → ROM → IR → Datapath.

در هر سیکل کلاک، آدرس موجود در PC محتوای متناظر حافظه برنامه را انتخاب می کند. این مقدار در IR ذخیره شده و شش بیت کم ارزش آن به مسیر داده داده می شود. در همان زمان، PC برای واکشی دستور بعدی افزایش می یابد. جدول ۱ اجزای اصلی مدار را نشان می دهد.

جزء	عرض	وظیفه
PC	۵ بیت	تولید آدرس ۳۲ خانه حافظه برنامه و افزایش ترتیبی آدرس
ROM	32×8	نگهداری کد ماشین برنامه
IR	۸ بیت	نگهداری دستور واکشی شده تا زمان اجرای آن
R0-R3	هر کدام ۸ بیت	ثبات های عمومی ماشین
ALU	۸ بیت	انجام عملیات جمع یا تفریق
Datapath	—	انتخاب عملوندها، اجرای عملیات و نوشتن نتیجه در ثبات مقصد

جدول ۱: اجزای اصلی معماری پیاده سازی شده در آزمایش هفتم

۳ قالب دستور و کدگذاری

قالب اصلی دستور همان قالب شش بیتی آزمایش ششم است. بیت پر ارزش نوع عملیات، دو بیت بعدی ثبات مقصد و سه بیت کم ارزش منبع عملوند دوم را مشخص می کنند:

معنا	نام میدان	بیت‌ها
0: جمع، 1: تفریق	Op	b_5
11:R3, 10:R2, 01:R1, 00:R0	Destination	b_4b_3
110:-1, 101:1, 100:0, 011:R3, 010:R2, 001:R1, 000:R0	Source	$b_2b_1b_0$

جدول ۲: قالب دستور شش‌بیتی ماشین

از آن‌جا که حافظه برنامه ۸ بیتی است، دو بیت بالایی هر کلمه حافظه در این آزمایش صفر در نظر گرفته شده‌اند و دستور شش‌بیتی در بیت‌های $IR[5:0]$ قرار می‌گیرد. در نتیجه:

$$IR[5] \rightarrow Op, \quad IR[4:3] \rightarrow Destination, \quad IR[2:0] \rightarrow Source.$$

۴ تغییر مسیر داده نسبت به آزمایش ششم

در شکل معماری آزمایش ششم، یکی از ورودی‌های ALU به‌صورت ثابت از محتوای R_0 گرفته شده بود و تنها ورودی دوم با میدان Source انتخاب می‌شد. با این حال، برنامه فیبوناچی ارائه‌شده در آزمایش هفتم از دستورهایی مانند $Add\ R1, R0$ استفاده می‌کند و نتیجه مورد انتظار برای این دستور به‌صورت

$$R_1 \leftarrow R_1 + R_0$$

در نظر گرفته شده است. اگر ورودی اول ALU همواره R_0 باشد، همین دستور به‌اشتباه

$$R_1 \leftarrow R_0 + R_0$$

را تولید می‌کند و دنباله فیبوناچی از مرحله‌ای که مقادیر R_0 و R_1 متفاوت می‌شوند، نادرست خواهد شد. به همین دلیل، در مسیر داده این آزمایش یک مالتی‌پلکسر چهاربه‌یک به ورودی اول ALU اضافه شد. انتخاب این مالتی‌پلکسر توسط میدان Destination انجام می‌شود؛ بنابراین مقدار فعلی ثبات مقصد به‌عنوان عملوند اول وارد ALU می‌شود. در نتیجه معنای دستورها به‌صورت طبیعی زیر خواهد بود:

$$Destination \leftarrow Destination\ Op\ Source.$$

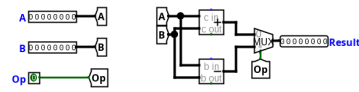
این تغییر نسبت به پیاده‌سازی آزمایش ششم باعث می‌شود برنامه ارائه‌شده در دستورکار آزمایش هفتم بدون تغییر و مطابق نتایج مورد انتظار اجرا شود.

۵ طراحی واحد محاسبات

واحد محاسبات از یک جمع‌کننده هشت‌بیتی، یک تفریق‌کننده هشت‌بیتی و یک مالتی‌پلکسر برای انتخاب نتیجه تشکیل شده است. دو ورودی A و B به‌طور هم‌زمان به هر دو مدار جمع و تفریق داده می‌شوند. سیگنال Op خروجی مناسب را انتخاب می‌کند:

$$Result = \begin{cases} A + B & Op = 0, \\ A - B & Op = 1. \end{cases}$$

شکل ۱ پیاده‌سازی این واحد را نشان می‌دهد.



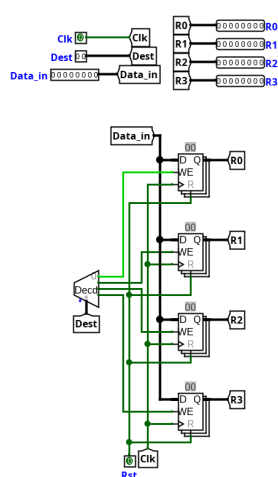
شکل ۱: پیاده‌سازی واحد جمع/تفریق هشت‌بیتی و انتخاب نتیجه توسط Op

۶ طراحی فایل ثبات

فایل ثبات شامل چهار ثبات عمومی هشت‌بیتی R_0 ، R_1 ، R_2 و R_3 است. ورودی داده هر چهار ثبات از خروجی ALU گرفته می‌شود، اما تنها ثباتی که توسط میدان Destination انتخاب شده است در لبه کلاک مقدار جدید را ذخیره می‌کند.

برای این منظور، میدان دو بیتی Destination به یک دیکدر دو به چهار داده شده و چهار خروجی دیکدر به ورودی Write Enable ثبات‌ها متصل شده‌اند. سیگنال‌های Clk و Rst نیز بین تمام ثبات‌ها مشترک هستند. با فعال شدن Rst، محتوای هر چهار ثبات صفر می‌شود.

شکل ۲ ساختار فایل ثبات را نشان می‌دهد.

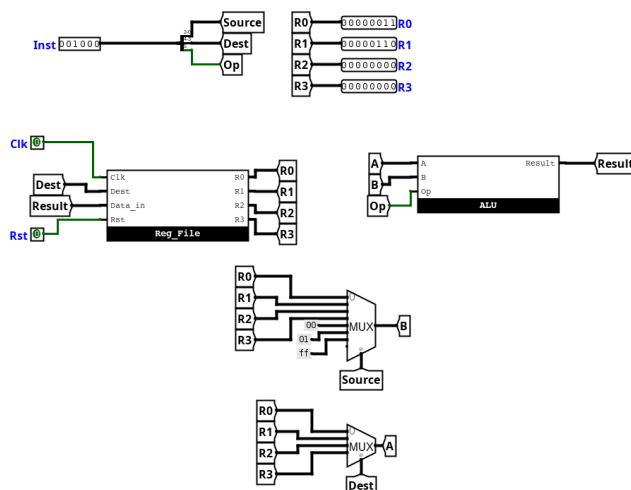


شکل ۲: پیاده‌سازی فایل ثبات شامل چهار ثبات عمومی هشت‌بیتی

۷ طراحی مسیر داده

در مسیر داده، دستور شش‌بیتی ابتدا توسط یک Splitter به سه میدان Source، Destination و Op تقسیم می‌شود. چهار خروجی فایل ثبات به دو مسیر انتخاب جداگانه متصل هستند. مالتی‌پلکسر اول با سیگنال Destination کنترل می‌شود و مقدار فعلی ثبات مقصد را برای ورودی انتخاب می‌کند. مالتی‌پلکسر دوم با سیگنال سه‌بیتی Source کنترل شده و یکی از مقادیر R_0 تا R_3 یا ثابت‌های 0، 1 و -1 را برای ورودی B انتخاب می‌کند. مقدار -1 در نمایش هشت‌بیتی مکمل دو برابر FF است.

پس از انجام عملیات در ALU، نتیجه به ورودی فایل ثبات بازگردانده می‌شود و در لبه کلاک در ثبات مقصد ذخیره می‌شود. شکل ۳ مسیر داده کامل را نشان می‌دهد.



شکل ۳: مسیر داده کامل شامل فایل ثبات، مالتی پلکسرهای انتخاب عملوند و ALU

۸ واحد واکنشی دستور

برای خودکارشدن اجرای دستورات، سه جزء PC، حافظه برنامه و IR به مدار اضافه شده‌اند.

۱.۸ شمارنده برنامه

حافظه برنامه شامل ۳۲ خانه است؛ بنابراین برای آدرس‌دهی آن به پنج بیت نیاز داریم:

$$2^5 = 32.$$

شمارنده برنامه یک شمارنده پنج‌بیتی است که پس از بازنشانی از آدرس صفر شروع کرده و با هر کلاک یک واحد افزایش می‌یابد. خروجی آن مستقیماً به باس آدرس حافظه برنامه متصل است.

۲.۸ حافظه برنامه

حافظه برنامه به صورت یک ROM با سازمان 32×8 پیاده‌سازی شده است. برنامه فیوناچی از آدرس 00 آغاز می‌شود و ده دستور اصلی در آدرس‌های 00 تا 09 قرار دارند. خانه‌های بعدی با کد 04 پر شده‌اند. این کد متناظر با دستور $Add\ R0, 0$ است و در مسیر داده اصلاح شده عملاً مقدار $R0$ را بدون تغییر در خود آن می‌نویسد؛ بنابراین پس از پایان برنامه، ادامه افزایش PC باعث تغییر نتیجه نمی‌شود.

۳.۸ ثبات دستور

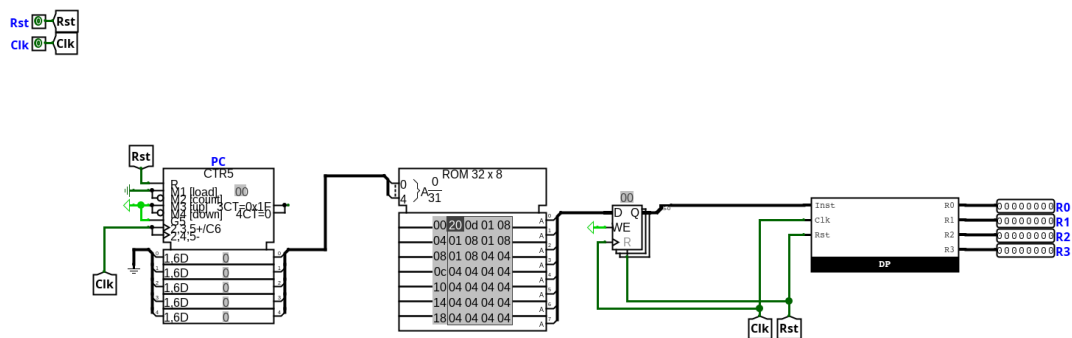
خروجی ۸ بیتی ROM ابتدا وارد ثبات دستور IR می‌شود. وجود IR باعث می‌شود دستور واکنشی شده در طول سیکل اجرا ثابت باقی بماند. شش بیت کم‌ارزش IR به ورودی Inst مسیر داده متصل شده‌اند و دو

بیت بالایی در این آزمایش استفاده نمی‌شوند.

با استفاده از کلاک مشترک، در هر لبه کلاک سه رخداد اصلی اتفاق می‌افتند: دستور فعلی موجود در IR توسط مسیر داده اجرا می‌شود، دستور جدید از ROM در IR قرار می‌گیرد و PC برای آدرس بعدی افزایش می‌یابد. به این ترتیب، پس از مرحله اولیه واکنشی، برنامه به صورت پیوسته و ترتیبی اجرا می‌شود.

۹ مدار نهایی

پس از تکمیل مسیر داده و واحد واکنشی دستور، مدار اصلی مطابق شکل ۴ ساخته شد. سیگنال‌های Clk و Rst بین اجزای ترتیبی مشترک‌اند. خروجی پنج‌بیتی PC به آدرس ROM متصل شده، خروجی ۸ بیتی حافظه وارد IR شده و شش بیت کم‌ارزش IR به مسیر داده داده می‌شود. خروجی‌های R_0 تا R_3 نیز برای مشاهده وضعیت اجرای برنامه در مدار اصلی نمایش داده شده‌اند.



شکل ۴: مدار نهایی آزمایش هفتم شامل PC، حافظه برنامه، IR و مسیر داده

۱۰ برنامه تولید سری فیبوناچی

در سری فیبوناچی دو جمله اول برابر 0 و 1 هستند و هر جمله بعدی از جمع دو جمله قبل به دست می‌آید. در دستورکار، در یک عبارت به تولید شش جمله اشاره شده است، اما جدول برنامه بلافاصله ده جمله اول را در ثبات‌های R_0 و R_1 تولید می‌کند. در این پیاده‌سازی، همان برنامه ده‌دستوری جدول دستورکار در نظر گرفته شده است.

دو دستور نخست مقداردهی اولیه را انجام می‌دهند:

$$R_0 \leftarrow 0, \quad R_1 \leftarrow 1.$$

پس از آن، جمع به صورت متناوب در R_0 و R_1 انجام می‌شود تا جمله‌های بعدی تولید شوند. جدول ۳ کدگذاری برنامه و وضعیت ثبات‌ها پس از اجرای هر دستور را نشان می‌دهد. دو بیت بالایی کلمه ۸ بیتی حافظه در تمام این دستورات صفر هستند.

آدرس	دستور	کد شش‌بیتی	کد هگز	R_0	R_1
00	Sub R_0, R_0	100000	20	۰	۰
01	Add $R_1, 1$	001101	0D	۰	۱
02	Add R_0, R_1	000001	01	۱	۱
03	Add R_1, R_0	001000	08	۱	۲
04	Add R_0, R_1	000001	01	۳	۲
05	Add R_1, R_0	001000	08	۳	۵
06	Add R_0, R_1	000001	01	۸	۵
07	Add R_1, R_0	001000	08	۸	۱۳
08	Add R_0, R_1	000001	01	۲۱	۱۳
09	Add R_1, R_0	001000	08	۲۱	۳۴

جدول ۳: برنامه ذخیره‌شده در حافظه برای تولید ده جمله اول سری فیبوناچی

۱۱ تست و شبیه‌سازی

برای بررسی صحت عملکرد مدار، ابتدا سیستم بازنشانی شد تا PC، IR و ثبات‌های عمومی در وضعیت اولیه قرار گیرند. سپس کلاک به صورت مرحله به مرحله اعمال شد و تغییرات PC، مقدار دستور در IR و محتوای R_0 و R_1 مشاهده شد.

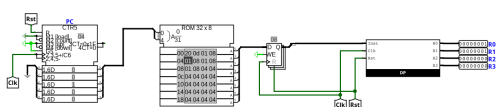
به جای استفاده از یک تصویر برای هر ده دستور، چهار وضعیت متوالی از ابتدای اجرای برنامه انتخاب شد تا روند صحیح واکنشی و اجرای دستورات مشخص باشد. در ادامه نیز یک تصویر جداگانه از پایان برنامه ارائه شده است. این روش هم توالی اجرای برنامه را نشان می‌دهد و هم از تکرار تعداد زیادی تصویر مشابه جلوگیری می‌کند.

۱.۱۱ روند اجرای دستورهای ابتدایی

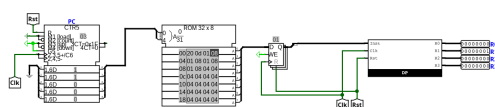
در وضعیت نخست، دو دستور مقداردهی اولیه اجرا شده‌اند و ثبات‌ها به مقادیر

$$R_0 = 0, \quad R_1 = 1$$

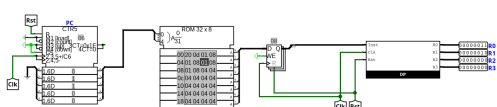
رسیده‌اند. سپس دستور Add R_0, R_1 اجرا شده و جمله سوم تولید می‌شود. در مرحله بعد، دستور Add R_1, R_0 جمله چهارم را تولید می‌کند و پس از آن اجرای مجدد Add R_0, R_1 مقدار 3 را در R_0 قرار می‌دهد. شکل ۵ چهار وضعیت متوالی این روند را نشان می‌دهد. مقدار نمایش داده شده در IR دستور آماده اجرا در همان مرحله است و مقادیر ثبات‌ها نتیجه دستورهای اجرا شده تا آن لحظه را نشان می‌دهند.



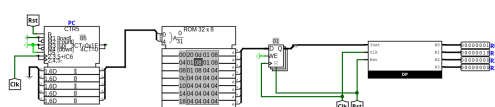
(ب) تولید جمله سوم و رسیدن به $R_1 = 1, R_0 = 1$



(آ) $R_0 = 0$ و $R_1 = 1$ پس از مقداردهی اولیه



(د) ادامه برنامه و تولید مقدار 3 در R_0



(ج) تولید جمله چهارم و رسیدن به $R_1 = 2, R_0 = 1$

شکل ۵: روند مرحله به مرحله اجرای بخش ابتدایی برنامه فیبوناچی

این چهار وضعیت نشان می دهند که PC به ترتیب افزایش می یابد، دستورات به ترتیب از حافظه واکنشی می شوند و نتیجه هر دستور در ثبات مقصد مناسب قرار می گیرد. همچنین مرحله ای که $R_0 = 3$ و $R_1 = 2$ می شود، صحت تغییر اعمال شده در مسیر داده را نیز تأیید می کند؛ زیرا در دستور بعدی $Add R_1, R_0$ باید دو مقدار متفاوت 2 و 3 با یکدیگر جمع شوند و حاصل 5 در R_1 قرار گیرد.

۲.۱۱ نتیجه نهایی برنامه

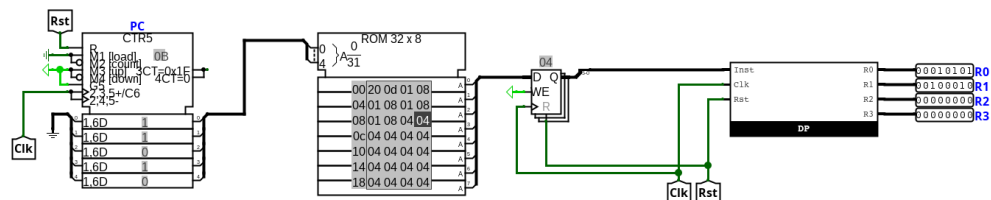
پس از اجرای هر ده دستور اصلی، دو ثبات فعال شامل دو جمله پایانی از ده جمله اول سری هستند:

$$R_0 = 21, \quad R_1 = 34.$$

نمایش باینری این مقادیر برابر است با:

$$21_{10} = 00010101_2, \quad 34_{10} = 00100010_2.$$

در شکل ۶ مقدار PC از محدوده ده دستور اصلی عبور کرده و IR حاوی کد 04 است. از آنجا که خانه های بعدی حافظه با دستور بدون اثر $Add R_0, 0$ پر شده اند، مقادیر نهایی $R_0 = 21$ و $R_1 = 34$ ثابت باقی می ماند.



شکل ۶: وضعیت نهایی پس از اجرای برنامه؛ $R_1 = 34$ و $R_0 = 21$

مطابقت تمام مقادیر جدول ۳ با نتایج شبیه‌سازی و رسیدن به مقادیر نهایی ۲۱ و ۳۴ نشان می‌دهد که واحد واکنشی دستور، ثبات دستور، مسیر داده و فایل ثبات به‌صورت هماهنگ عمل می‌کنند.

۱۲ نتیجه‌گیری

در این آزمایش، واحد محاسبات ساخته‌شده در آزمایش ششم به یک سیستم با قابلیت اجرای برنامه ذخیره‌شده در حافظه توسعه داده شد. برای این منظور، یک شمارنده برنامه پنج‌بیتی، حافظه برنامه 8×32 و ثبات دستور هشت‌بیتی به مدار اضافه شدند. با این ساختار، دستورات دیگر به‌صورت دستی به مسیر داده اعمال نمی‌شوند و پردازنده می‌تواند آن‌ها را به‌ترتیب از حافظه واکنشی و اجرا کند.

در مسیر داده نیز نسبت به آزمایش ششم یک تغییر انجام شد. برای سازگاری با معنای دستورهای برنامه ارائه‌شده در آزمایش هفتم، عملوند اول ALU به‌جای اتصال ثابت به R_0 ، از طریق یک مالتی‌پلکسر از ثبات مقصد انتخاب شد. در نتیجه، دستورهای مانند R_1, R_0 Add به‌درستی عملیات $R_1 \leftarrow R_1 + R_0$ را انجام می‌دهند.

برنامه ده‌دستوری تولید سری فیبوناچی در حافظه برنامه قرار گرفت و اجرای مرحله‌به‌مرحله آن بررسی شد. مقادیر ثبات‌ها به‌ترتیب با دنباله مورد انتظار منطبق بودند و پس از پایان برنامه، مقادیر $R_0 = 21$ و $R_1 = 34$ به‌دست آمدند. همچنین خانه‌های استفاده‌نشده حافظه با دستور $R_0, 0$ Add پر شدند تا ادامه شمارش PC پس از پایان برنامه باعث تغییر نتیجه نشود.

نتایج شبیه‌سازی نشان دادند که فرآیند واکنشی دستور از حافظه، نگهداری آن در IR، رمزگشایی میدان‌های دستور، اجرای عملیات در مسیر داده و ذخیره نتیجه در ثبات مقصد مطابق انتظار انجام می‌شوند.